# shared_ptr — Խորը ուղեցույց

### C++ Smart Pointer presentation

Այս ուղեցույցը մանրամասն բացatrum e `std::shared_ptr`-y՝ inch e, reference counting, control block, make_shared, use_count, circular reference problem, orinaknerov ev interview harcerov.

---

# 1. Ի՞նչ է shared_ptr-ը

`std::shared_ptr`-y smart pointer e **shared ownership**-ov. Mek resource-y karox e patkanel **mi qani** shared_ptr-i miasin. Resource-y azatvum e, erb **verjin** shared_ptr-y ochncum e.

```
#include <memory>

auto p1 = std::make_shared<int>(10);
auto p2 = p1;               // OK -- copy, erku-n el owner
cout << *p1 << *p2;         // 10 10
// resource azatvum e erb U p1 U p2 ochncvum
```

---

# 2. Reference Counting

shared_ptr-y pahum e **hashvich** (reference count)՝ qani owner ka.

```cpp
auto p1 = std::make_shared<int>(10);
cout << p1.use_count();     // 1

auto p2 = p1;
cout << p1.use_count();     // 2

{
    auto p3 = p1;
    cout << p1.use_count(); // 3
}                           // p3 ochncum -> count 2

cout << p1.use_count();     // 2
```

```
copy       -> count++
ochncum    -> count--
count == 0 -> resource DELETE
```

---

# 3. Control Block

shared_ptr-y ners uni **erku ban**.

```
1. Pointer -> resource-in (tvyaln)
2. Pointer -> control block-in
     control block:
       - reference count (strong)
       - weak count
       - deleter
```

```
p1 --+
     +--> [Control Block: strong=2, weak=0] --> [ Resource: 10 ]
p2 --+
```

Amen copy -> nuyn control block, strong count++. Sa aŕancin memory e (dra hamar make_shared-y aveli lav e` mek allocation).

---

# 4. make_shared (naxentreli)

```
auto p = std::make_shared<int>(10);          // 1 allocation (resource + control block miasin)
// vs
std::shared_ptr<int> q(new int(10));         // 2 allocation (aveli dandaghn)
```

`make_shared` -y resource-y U control block-y **mek allocation-ov** e steghcum -> aveli arag, exception-safe.

---

# 5. Karevor metodner

```
auto p = std::make_shared<int>(10);

*p;                // dereference
p.get();           // raw pointer
p.use_count();     // qani owner
p.reset();         // count--, ete verjiny -> delete
p.unique();        // count == 1? (deprecated C++17)
if (p) { }         // null che?
```

---

# 6. Circular Reference — MEC XNDIR

shared_ptr-i amenamec vtangy` **circular reference** (erku object cuyc en talis irar) -> count erbek 0 chi darnum -> LEAK.

```
struct Node {
    std::shared_ptr<Node> next;
};

auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;    // b-i count = 2
b->next = a;    // a-i count = 2
// scope verj: a,b ochncvum -> count 1 (irar pahum) -> ERBEK 0 -> LEAK
```

```
a --> [Node A] --> b
b --> [Node B] --> a
   circular -> count erbek 0 chi darnum -> LEAK
```

### Lucumy` weak_ptr

```
struct Node {
    std::weak_ptr<Node> next;   // weak -> count chi achecnum
};
```

weak_ptr-y **chi achecnum strong count-y** -> circular kotrvum -> chka leak. (Tes weak_ptr presentation-y.)

---

## 7. Overhead

```
shared_ptr:
  - unique_ptr-ic AVELI canr (control block, atomic count)
  - reference count-y ATOMIC e (thread-safe count) -> mi qich dandaghn
  - 2 pointer-i chapov (resource + control block)
```

Ogtagorci shared_ptr **miayn erb iskapes shared ownership petq e**; ayl depqum unique_ptr.

---

## 8. Interview harcer & patasxanner

**shared_ptr inch e?**

> *Smart pointer shared ownership-ov; mi qani owner; verjin owner-i ochncelis resource delete (reference counting).*

**Reference counting inchpes?**

> *copy -> count++, ochncum -> count--, count==0 -> delete.*

**Control block inch e?**

> *Ańcin block strong count + weak count + deleter-ov; bolor copy-nery nuyn control block.*

**make_shared vs shared_ptr(new)?**

> *make_shared 1 allocation (arag, exception-safe); (new) 2 allocation.*

**Circular reference inch e?**

> *Erku shared_ptr irar cuyc talis -> count erbek 0 -> leak. Lucum ` weak_ptr.*

**use_count() inch e?**

> *Qani strong owner ka.*

**shared vs unique overhead?**

> *shared aveli canr (atomic count, control block); unique zero overhead.*

---

# Cheat Sheet

```
shared_ptr = SHARED ownership (mi qani owner), reference counting

  make_shared<T>(args)  -> steghcum (1 allocation, NAXENTRELI)
  copy -> count++ | ochncum -> count-- | count==0 -> DELETE
  use_count()  -> qani owner
  get() / reset() / *p / p->

  CONTROL BLOCK: strong count + weak count + deleter (atomic)

  CIRCULAR REFERENCE -> count erbek 0 -> LEAK -> lucum` weak_ptr

  overhead: unique-ic canr (atomic count) -> ogtagorci miayn erb shared petq e
```